Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №7
на тему
«ЗАЩИТА ПО ОТ НЕСАНКЦИОНИРОВАННОГО ИСПОЛЬЗОВАНИЯ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Теоретические сведения 4
2.2 Реализация обфускатора 4
2.3 Результаты тестирования 5
Вывод 6
Приложение А (обязательное) Листинг программного кода 7

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение методов обфускации исходного кода программного обеспечения. В рамках работы необходимо разработать утилиту для обфускации программ, а также продемонстрировать её работу на примере тестового приложения.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Теоретические сведения

Обфускация – это процесс приведения исходного или исполняемого кода программы к виду, сохраняющему её функциональность, но затрудняющему анализ, понимание алгоритмов работы и модификацию. Обфускация применяется для защиты интеллектуальной собственности, усложнения реверс-инжиниринга и предотвращения несанкционированного использования кода.

Основными методами обфускации на уровне исходного кода являются:

– лексическая обфускация (переименование переменных, функций, классов в бессмысленные имена);

– обфускация данных (шифрование строк, кодирование числовых констант);

– обфускация управления (добавление мертвого кода, изменение графа потока управления);

– удаление метаданных (комментариев, отладочной информации).

В языке *Go* для анализа и модификации исходного кода предоставляются стандартные пакеты, такие как *go/ast*, *go/parser* и *go/printer*, которые позволяют строить абстрактное синтаксическое дерево (*AST*), изменять его узлы и генерировать измененный код.

2.2 Реализация обфускатора

Для выполнения поставленной задачи была разработана программа-обфускатор на языке *Go*. Программа выполняет две основные функции: переименование идентификаторов и обфускацию строковых литералов.

Для генерации новых имен идентификаторов используется алгоритм, создающий случайные строки, состоящие из визуально похожих символов: „I“, „l“, „O“, „1“, „0“. Первый символ всегда выбирается из букв „I“, „l“, „O“, чтобы соответствовать правилам именования в языке *Go*. Для сохранения консистентности используется словарь (*map*), который ставит в соответствие каждому оригинальному имени сгенерированное новое.

Обфускация строк реализована с помощью операции *XOR* с фиксированным ключом (в данном случае `0xAA`). Программа обходит *AST* исходного файла, находит все вызовы функций со строковыми аргументами и заменяет строковый литерал на вызов функции дешифрования, передавая ей массив зашифрованных байт.

Для корректной работы модифицированной программы обфускатор генерирует и внедряет в целевой файл функцию дешифрования. Эта функция принимает массив зашифрованных байт, применяет к каждому байту операцию *XOR* с ключом и возвращает расшифрованную строку.

2.3 Результаты тестирования

Для демонстрации работы обфускатора была использована программа из предыдущей лабораторной работы, реализующая безопасную и небезопасную запись в буфер.

Сначала был осуществлен запуск оригинальной программы, которая корректно вывела результаты работы функций записи. Затем исходный код программы был обработан разработанным обфускатором. Полученный модифицированный файл был успешно скомпилирован и запущен.

На рисунке 2.1 представлены результаты выполнения оригинальной и обфусцированной программ, которые показывают идентичность их работы.

```
velebes@arch ... (main)> go run lab_to_obfuscate/main.go
Before write:
buf1: "AAAAAAAA"
buf2: "BBBBBBBB"

Enter your message:
helloaboabasad
Input (14 bytes): "helloaboabasad"

Unsafe write to buf1:
buf1: "helloabo"
buf2: "abasadBB"

Safe write to buf1:
error: truncated: wrote 8 of 14 bytes
buf1: "helloabo"
buf2: "BBBBBBBB"
```

```
velebes@arch ... (main)> go run lab_to_obfuscate/main_obfuscated.go
Before write:
buf1: "AAAAAAAA"
buf2: "BBBBBBBB"

Enter your message:
helloaboabasad
Input (14 bytes): "helloaboabasad"

Unsafe write to buf1:
buf1: "helloabo"
buf2: "abasadBB"

Safe write to buf1:
error: truncated: wrote 8 of 14 bytes
buf1: "helloabo"
buf2: "BBBBBBBB"
```

Рисунок 2.1 – Результаты тестирования оригинальной и обфусцированной программ

Исходный код обфусцированной программы стал трудночитаемым из-за замены имен функций и переменных на визуально неразличимые последовательности символов, а также замены всех удобочитаемых строк на массивы байт. Это подтверждает успешное применение лексической обфускации и обфускации данных.

ВЫВОД

В ходе выполнения лабораторной работы были изучены методы запутывания исходного кода программного обеспечения. Разработана утилита на языке *Go*, использующая пакеты для работы с абстрактным синтаксическим деревом (*AST*) для модификации исходного кода.

Реализованный обфускатор успешно выполняет лексическую обфускацию путем замены осмысленных имен идентификаторов на визуально трудноразличимые строки, а также обфускацию данных посредством применения операции *XOR* к строковым константам. Процесс обфускации автоматизирован: программа самостоятельно находит необходимые узлы в *AST*, модифицирует их и внедряет необходимую функцию для расшифровки строк во время выполнения.

Тестирование разработанного обфускатора на целевом приложении подтвердило, что после обработки исходного кода функциональность программы полностью сохраняется, при этом сам код становится сложным для анализа и понимания. Это демонстрирует эффективность примененных методов защиты кода от реверс-инжиниринга.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Код обфускатора

```
package main

import (
  "fmt"
  "go/ast"
  "go/parser"
  "go/printer"
  "go/token"
  "math/rand"
  "os"
  "strconv"
)

type Obfuscator struct {
  mapping map[string]string
  fset    *token.FileSet
  key     byte
}

func NewObfuscator() *Obfuscator {
  return &Obfuscator{
    mapping: make(map[string]string),
    fset:    token.NewFileSet(),
    key:     0xAA,
  }
}

func (o *Obfuscator) generateNewName(oldName string) string {
  if newName, ok := o.mapping[oldName]; ok {
    return newName
  }

  noDigits := []rune("IlO")
  chars := []rune("10IlO")
  res := make([]rune, 12)
  res[0] = noDigits[rand.Intn(len(noDigits))]

  for i := 1; i < len(res); i++ {
    res[i] = chars[rand.Intn(len(chars))]
  }
  o.mapping[oldName] = string(res)
  return string(res)
}

func (o *Obfuscator) collectIdentifiers(node *ast.File) {
  ast.Inspect(node, func(n ast.Node) bool {
    switch x := n.(type) {
    case *ast.StructType:
      for _, field := range x.Fields.List {
```

```
        for _, name := range field.Names {
          o.generateNewName(name.Name)
        }
      }
    case *ast.FuncDecl:
      if x.Name.Name != "main" {
        o.generateNewName(x.Name.Name)
      }
    }
    return true
  })
}

func (o *Obfuscator) createDecryptCall(val string) *ast.CallExpr {
  elements := []ast.Expr{}
  for j := 0; j < len(val); j++ {
    elements = append(elements, &ast.BasicLit{
      Kind:  token.INT,
      Value: fmt.Sprintf("0x%02x", val[j]^o.key),
    })
  }
  return &ast.CallExpr{
    Fun: &ast.Ident{Name: o.generateNewName("decode")},
    Args: []ast.Expr{
      &ast.CompositeLit{
        Type: &ast.ArrayType{
          Elt: &ast.Ident{Name: "byte"},
        },
        Elts: elements,
      },
    },
  }
}

func (o *Obfuscator) obfuscateIdentifiersAndStrings(node *ast.File) {
  ast.Inspect(node, func(n ast.Node) bool {
    switch x := n.(type) {
    case *ast.Ident:
      if newName, ok := o.mapping[x.Name]; ok {
        x.Name = newName
        return true
      }
      if x.Name != "main" && x.Obj != nil && x.Name != "d" {
        x.Name = o.generateNewName(x.Name)
      }

    case *ast.KeyValueExpr:
      if key, ok := x.Key.(*ast.Ident); ok {
        if newName, ok := o.mapping[key.Name]; ok {
          key.Name = newName
        }
      }

    case *ast.CallExpr:
      for i, arg := range x.Args {
      if lit, ok := arg.(*ast.BasicLit); ok && lit.Kind == token.STRING {
          val, err := strconv.Unquote(lit.Value)
```

```
          if err != nil || len(val) == 0 {
            continue
          }
          x.Args[i] = o.createDecryptCall(val)
        }
      }
    }
    return true
  })
}

func (o *Obfuscator) injectDecryptFunc(node *ast.File) {
  decryptFunc := &ast.FuncDecl{
    Name: &ast.Ident{Name: o.generateNewName("decode")},
    Type: &ast.FuncType{
      Params: &ast.FieldList{
        List: []*ast.Field{{
          Names: []*ast.Ident{{Name: o.generateNewName("b")}},
          Type:  &ast.ArrayType{Elt: &ast.Ident{Name: "byte"}},
        }},
      },
      Results: &ast.FieldList{
        List: []*ast.Field{{Type: &ast.Ident{Name: "string"}}},
      },
    },
    Body: &ast.BlockStmt{
      List: []ast.Stmt{
        &ast.RangeStmt{
          Key: &ast.Ident{Name: o.generateNewName("i")},
          Tok: token.DEFINE,
          X:   &ast.Ident{Name: o.generateNewName("b")},
          Body: &ast.BlockStmt{
            List: []ast.Stmt{
              &ast.AssignStmt{
                                    Lhs:  []ast.Expr{&ast.IndexExpr{X:
&ast.Ident{Name:   o.generateNewName("b")},   Index:   &ast.Ident{Name:
o.generateNewName("i")}}},
                Tok: token.XOR_ASSIGN,
                  Rhs: []ast.Expr{&ast.BasicLit{Kind: token.INT, Value:
fmt.Sprintf("0x%02x", o.key)}},
              },
            },
          },
        },
        &ast.ReturnStmt{
          Results: []ast.Expr{
                  &ast.CallExpr{Fun:  &ast.Ident{Name:  "string"}, Args:
[]ast.Expr{&ast.Ident{Name: o.generateNewName("b")}}},
          },
        },
      },
    },
  }
  node.Decls = append(node.Decls, decryptFunc)
}

func (o *Obfuscator) Obfuscate(inputPath, outputPath string) error {
```

```
      node,   err   :=   parser.ParseFile(o.fset,   inputPath,   nil,
parser.ParseComments)
  if err != nil {
    return err
  }

  o.collectIdentifiers(node)
  o.obfuscateIdentifiersAndStrings(node)
  o.injectDecryptFunc(node)

  f, err := os.Create(outputPath)
  if err != nil {
    return err
  }
  defer f.Close()

  return printer.Fprint(f, o.fset, node)
}

func main() {
  obfs := NewObfuscator()
  err := obfs.Obfuscate("lab_to_obfuscate/main.go", "lab_to_obfuscate/
main_obfuscated.go")
  if err != nil {
    fmt.Printf("Error: %v\n", err)
    return
  }
  fmt.Println("Obfuscation complete!")
}
```

Листинг А.2 – Код целевой программы до обфускации

```
package main

import (
  "fmt"
  "unsafe"
)

type Buffers struct {
  buf1 [8]byte
  buf2 [8]byte
}

func main() {
  b := Buffers{
    buf1: [8]byte{'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A'},
    buf2: [8]byte{'B', 'B', 'B', 'B', 'B', 'B', 'B', 'B'},
  }

  fmt.Println("Before write: ")
  fmt.Printf("buf1: %q\n", string(b.buf1[:]))
  fmt.Printf("buf2: %q\n", string(b.buf2[:]))
  fmt.Println()

  var input []byte
```

```
  fmt.Println("Enter your message:")
  fmt.Scan(&input)
  fmt.Printf("Input (%d bytes): %q\n\n", len(input), string(input))

  fmt.Println("Unsafe write to buf1:")
  unsafeWrite(&b.buf1, input)
  fmt.Printf("buf1: %q\n", string(b.buf1[:]))
  fmt.Printf("buf2: %q\n", string(b.buf2[:]))
  fmt.Println()

  b = Buffers{
    buf1: [8]byte{'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A'},
    buf2: [8]byte{'B', 'B', 'B', 'B', 'B', 'B', 'B', 'B'},
  }

  fmt.Println("Safe write to buf1:")
  err := safeWrite(&b.buf1, input)
  if err != nil {
    fmt.Printf("error: %v\n", err)
  }
  fmt.Printf("buf1: %q\n", string(b.buf1[:]))
  fmt.Printf("buf2: %q\n", string(b.buf2[:]))
}

func unsafeWrite(buf *[8]byte, data []byte) {
  for i := range data {
    ptr := (*byte)(unsafe.Add(unsafe.Pointer(buf), i))
    *ptr = data[i]
  }
}

func safeWrite(buf *[8]byte, data []byte) error {
  var err error
  n := len(data)
  if n > len(buf) {
    n = len(buf)
    err = fmt.Errorf("truncated: wrote %d of %d bytes", n, len(data))
  }
  for i := 0; i < n; i++ {
    buf[i] = data[i]
  }
  return err
}
```

Листинг А.3 – Код целевой программы после обфускации

```
package main

import (
  "fmt"
  "unsafe"
)

type O01lO0I1l10O struct {
  O0OI1I110l1O  [8]byte
  OllI00Ol1l0O  [8]byte
```

```
}

func main() {
  OOl0I0lO0lOI := O01lO0I1l10O{
    O0OI1I110l1O: [8]byte{'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A'},
    OllI000l1l0O: [8]byte{'B', 'B', 'B', 'B', 'B', 'B', 'B', 'B'},
  }

   fmt.Println(OIlOIl0100II([]byte{0xe8, 0xcf, 0xcc, 0xc5, 0xd8, 0xcf,
0x8a, 0xdd, 0xd8, 0xc3, 0xde, 0xcf, 0x90, 0x8a}))
   fmt.Printf(OIlOIl0100II([]byte{0xc8, 0xdf, 0xcc, 0x9b, 0x90, 0x8a,
0x8f, 0xdb, 0xa0}), string(OOl0I0lO0lOI.O0OI1I110l1O[:]))
   fmt.Printf(OIlOIl0100II([]byte{0xc8, 0xdf, 0xcc, 0x98, 0x90, 0x8a,
0x8f, 0xdb, 0xa0}), string(OOl0I0lO0lOI.OllI000l1l0O[:]))
  fmt.Println()

  var I01llIII0O00 []byte
   fmt.Println(OIlOIl0100II([]byte{0xef, 0xc4, 0xde, 0xcf, 0xd8, 0x8a,
0xd3, 0xc5, 0xdf, 0xd8, 0x8a, 0xc7, 0xcf, 0xd9, 0xd9, 0xcb, 0xcd, 0xcf,
0x90}))
  fmt.Scan(&I01llIII0O00)
   fmt.Printf(OIlOIl0100II([]byte{0xe3, 0xc4, 0xda, 0xdf, 0xde, 0x8a,
0x82, 0x8f, 0xce, 0x8a, 0xc8, 0xd3, 0xde, 0xcf, 0xd9, 0x83, 0x90, 0x8a,
0x8f, 0xdb, 0xa0, 0xa0}), len(I01llIII0O00), string(I01llIII0O00))

   fmt.Println(OIlOIl0100II([]byte{0xff, 0xc4, 0xd9, 0xcb, 0xcc, 0xcf,
0x8a, 0xdd, 0xd8, 0xc3, 0xde, 0xcf, 0x8a, 0xde, 0xc5, 0x8a, 0xc8, 0xdf,
0xcc, 0x9b, 0x90}))
  I1Il1111O1Ol(&OOl0I0lO0lOI.O0OI1I110l1O, I01llIII0O00)
   fmt.Printf(OIlOIl0100II([]byte{0xc8, 0xdf, 0xcc, 0x9b, 0x90, 0x8a,
0x8f, 0xdb, 0xa0}), string(OOl0I0lO0lOI.O0OI1I110l1O[:]))
   fmt.Printf(OIlOIl0100II([]byte{0xc8, 0xdf, 0xcc, 0x98, 0x90, 0x8a,
0x8f, 0xdb, 0xa0}), string(OOl0I0lO0lOI.OllI000l1l0O[:]))
  fmt.Println()

  OOl0I0lO0lOI = O01lO0I1l10O{
    O0OI1I110l1O: [8]byte{'A', 'A', 'A', 'A', 'A', 'A', 'A', 'A'},
    OllI000l1l0O: [8]byte{'B', 'B', 'B', 'B', 'B', 'B', 'B', 'B'},
  }

   fmt.Println(OIlOIl0100II([]byte{0xf9, 0xcb, 0xcc, 0xcf, 0x8a, 0xdd,
0xd8, 0xc3, 0xde, 0xcf, 0x8a, 0xde, 0xc5, 0x8a, 0xc8, 0xdf, 0xcc, 0x9b,
0x90}))
  Il1lO0OOl011 := O1I11O00I1I1(&OOl0I0lO0lOI.O0OI1I110l1O, I01llIII0O00)
  if Il1lO0OOl011 != nil {
     fmt.Printf(OIlOIl0100II([]byte{0xcf, 0xd8, 0xd8, 0xc5, 0xd8, 0x90,
0x8a, 0x8f, 0xdc, 0xa0}), Il1lO0OOl011)
  }
   fmt.Printf(OIlOIl0100II([]byte{0xc8, 0xdf, 0xcc, 0x9b, 0x90, 0x8a,
0x8f, 0xdb, 0xa0}), string(OOl0I0lO0lOI.O0OI1I110l1O[:]))
   fmt.Printf(OIlOIl0100II([]byte{0xc8, 0xdf, 0xcc, 0x98, 0x90, 0x8a,
0x8f, 0xdb, 0xa0}), string(OOl0I0lO0lOI.OllI000l1l0O[:]))
}

func I1Il1111O1Ol(Ol0l00lO1000 *[8]byte, l101OlI0lll1 []byte) {
  for ll100lIl0OO1 := range l101OlI0lll1 {
      lO0II0OIOOOl  :=  (*byte)(unsafe.Add(unsafe.Pointer(Ol0l00lO1000),
```

```
ll100lIl0OO1))
    *lO0II0OIOOOl = l101OlI0ll1l[ll100lIl0OO1]
  }
}

func O1I11O00I1I1(Ol0l00lO1000 *[8]byte, l101OlI0ll1l []byte) error {
  var Il11lO0OOl011 error
  l11II0O0lOlO := len(l101OlI0ll1l)
  if l11II0O0lOlO > len(Ol0l00lO1000) {
    l11II0O0lOlO = len(Ol0l00lO1000)
   Il11lO0OOl011 = fmt.Errorf(OIlOIl0100II([]byte{0xde, 0xd8, 0xdf, 0xc4,
0xc9, 0xcb, 0xde, 0xcf, 0xce, 0x90, 0x8a, 0xdd, 0xd8, 0xc5, 0xde, 0xcf,
0x8a, 0x8f, 0xce, 0x8a, 0xc5, 0xcc, 0x8a, 0x8f, 0xce, 0x8a, 0xc8, 0xd3,
0xde, 0xcf, 0xd9}), l11II0O0lOlO, len(l101OlI0ll1l))
  }
  for ll100lIl0OO1 := 0; ll100lIl0OO1 < l11II0O0lOlO; ll100lIl0OO1++ {
    Ol0l00lO1000[ll100lIl0OO1] = l101OlI0ll1l[ll100lIl0OO1]
  }
  return Il11lO0OOl011
}
func OIlOIl0100II(OOl0I0lO0lOI []byte) string {
  for ll100lIl0OO1 := range OOl0I0lO0lOI {
    OOl0I0lO0lOI[ll100lIl0OO1] ^= 0xaa
  }
  return string(OOl0I0lO0lOI)
}
```